YASHAS RATTEHALLI

TECHNICAL SPECIFICATION

The Enterprise ERP Extraction

Resilient browser automation of a dynamic Odoo ERP — network-intercepted data extraction with strict, in-memory PHI sanitization and HIPAA-aligned controls.

CONFIDENTIAL — PREPARED FOR EVALUATION

Yashas Rattehalli
Senior AI Automation Engineer

Version 1.0.0
May 29, 2026
Verified: Odoo SaaS 19.3

Contents

1	Executive Summary	3
2	Challenge & Scope	4
3	Solution Architecture	5
4	Target Environment & Data Setup	6
5	Automation Workflow	7
6	Resilient Locator Strategy	8
7	Network Interception & No-Sleep Synchronization	9
8	Privacy Engineering: In-Memory PHI Sanitization	11
9	HIPAA Compliance Mapping	13
10	Data Model & Output Schema	15
11	Integrity & Verification	17
12	Operations & Runbook	18
13	Limitations, Assumptions & Future Work	19
A	Appendix A — Verified Run Evidence	20
B	Appendix B — References	21

1. Executive Summary

This document specifies a production-grade automation that authenticates to a live Odoo ERP, navigates its highly reactive Invoicing interface, isolates posted customer invoices, drills into a named customer's invoice, and extracts the billing lines into a clean, verifiable JSON document — while treating all billing data as Protected Health Information (PHI).

The implementation deliberately satisfies three engineering disciplines that distinguish robust enterprise automation from a brittle script: **(i) signal-based synchronization** with no hardcoded waits, **(ii) resilient, semantic locators** that survive the framework's ephemeral DOM, and **(iii) authoritative data capture via network interception** rather than fragile screen-scraping. Layered over these is a **privacy-by-design** posture: PHI exists only in transient memory, never reaches a log sink, and the sole file artifact is the final extract.

Target instance	Odoo SaaS 19.3 (odoo.com trial)
Workflow	Auth → Filter (Posted) → Locate → Drill → Extract → Sanitize
Data source	JSON-RPC `account.move/web_read` (server-computed)
Target customer	Deco Addict (configurable; alias-aware)
Result	3 lines · header/line totals reconciled · integrity fingerprint
Privacy	0 PHI tokens in logs · optional Safe Harbor de-identification

> **OUTCOME**
>
> A single command produces a standardized JSON extract whose monetary figures originate from the ERP's own computations, are independently reconciled, and are accompanied by a tamper-evident checksum — with a verifiable guarantee that no customer or product identifier ever appears in application logs.

2. Challenge & Scope

The objective is to demonstrate Playwright expertise in navigating complex, asynchronous enterprise applications (mimicking Electronic Health Record systems), authoring resilient locators, and implementing strict in-memory data sanitization. The mandated workflow is:

1. **Authentication & Navigation** — launch a persistent browser and reach the Invoicing module.
2. **Bypass dynamic UI** — clear default filters and apply the Posted status filter.
3. **Deep drill-down** — find the invoice for customer Deco Addict and open the detail view.
4. **Data extraction** — read the Invoice Lines (product, quantity, unit price, tax amount) into a standardized JSON array.

2.1 Hard constraints

Constraint	Specification & how it is honored
No hardcoded sleeps	`page.wait_for_timeout()` / `time.sleep()` are disqualifying. The automation uses only network-response triggers and auto-retrying assertions — see §7.
Resilient locators	Odoo's OWL framework emits ephemeral ids; we bind to semantic `name` attributes, ARIA roles, and stable framework classes — see §6.
Privacy guardrails	Billing data is treated as PHI. Logs never print the customer name or product details; PHI is scrubbed in memory and only the final JSON is written to disk — see §8.

The deliverables are this specification and the working Python Playwright script that executes the workflow and demonstrates the in-memory sanitization.

3. Solution Architecture

The system is a small, cohesive Python package with clearly separated responsibilities. Control flows left-to-right; the authoritative data path is the intercepted JSON-RPC response.

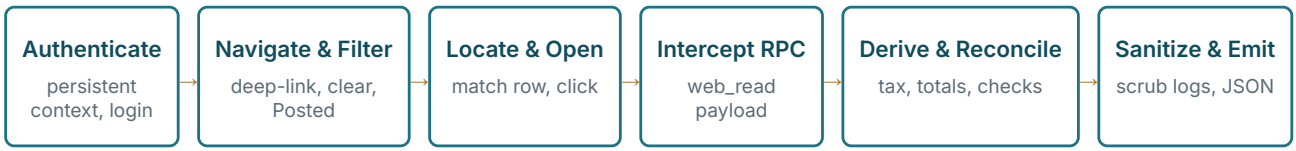

Figure 1 — End-to-end data flow. The PHI sanitization layer wraps every stage that emits a log record.

3.1 Components

Module	Responsibility
`cli.py`	Argument parsing, configuration assembly, logging bootstrap, exit codes.
`config.py`	Validated settings (pydantic-settings); secrets wrapped in `SecretStr`.
`odoo_client.py`	Playwright session; login, navigation, facet clearing, Posted filter, drill-down, DOM fallback.
`interception.py`	JSON-RPC predicates and payload parsing; per-line tax derivation; relational-value coercion.
`sanitization.py`	In-memory PHI redaction engine and the `logging.Filter`.
`logging_setup.py`	Handler-level sanitized logging configuration.
`deidentify.py`	HIPAA Safe Harbor de-identification pass.
`integrity.py`	SHA-256 line fingerprint; header/line reconciliation.
`models.py`	Typed output schema (Pydantic); computed `tax_amount`.
`extractor.py`	Orchestrates the workflow into a typed `ExtractionResult`.

4. Target Environment & Data Setup

The tool is **instance-agnostic**: it targets any Odoo instance via configuration and auto-detects the active URL scheme (modern `/odoo/…` vs legacy `/web#…`). It was verified against a freshly provisioned odoo.com trial running Odoo SaaS 19.3 with the Sales and Invoicing applications.

4.1 Demo data (Part 1)

A fresh trial renders client-side sample rows in an empty list view; these are placeholders, not records. To exercise the workflow faithfully, a small, idempotent helper (`scripts/seed_demo_data.py`) provisions a controlled dataset through Odoo's admin API:

- products with sales taxes (a standard 19% rate and a reduced 7% rate);
- customers — including **Deco Addict** — plus Azure Interior and Gemini Furniture;
- a mix of **posted and draft** invoices with multi-line, multi-tax content.

> **WHY THIS MATTERS**
>
> Deco Addict has both a posted and a draft invoice, so the Posted filter is genuinely required to select the correct record; and the posted invoice mixes 19% and 7% lines, so the per-line tax derivation is non-trivial and demonstrable.

> **VERSION NOTE — "DECO ADDICT"**
>
> Odoo's stock demo customer `res_partner_2` was named Deco Addict through v15 and relabelled Acme Corporation in v16+. The tool therefore treats the customer name as configuration and tries configured aliases in order — so the literal challenge target works regardless of the instance's demo-data vintage.

5. Automation Workflow

The orchestration (`extractor.run_extraction`) executes the mandated sequence. Each step is bounded by an explicit readiness signal.

#	Step	Behavior & readiness signal
1	Authenticate	Persistent context; submit credentials at `/web/login`; assert the application shell is visible. Skips the form when an existing session is detected.
2	Navigate	Deep-link to the `account.action_move_out_invoice_type` action; fall back across URL schemes; assert the list renderer is visible.
3	Clear filters	Remove any pre-applied facets, waiting on an auto-retrying facet-count assertion after each removal.
4	Apply Posted	Open the search dropdown; click the Posted filter; **await the resulting `search_read`**; assert the Posted facet is present.
5	Locate customer	Match the row whose `invoice_partner_display_name` cell contains the target (or an alias).
6	Open invoice	Click the row **inside** `expect_response(account.move/web_read)` so the authoritative payload is captured; assert the form view; confirm the status bar reads `posted`.
7	Extract lines	Parse the captured payload's nested line records; derive tax; reconcile; serialize.

6. Resilient Locator Strategy

Odoo's OWL framework regenerates DOM nodes and ids on each reactive update. Locators are therefore ranked by stability and never bound to generated artifacts. Element handles are never cached — every read re-queries the live DOM.

1. **Semantic field attributes** — `[name='<field>']`, rendered from OWL `t-att-name`, equal to the model field and stable across versions and languages.
2. **ARIA roles / accessible names** — e.g. `get_by_role('menuitemcheckbox', name='Posted')`.
3. **Stable framework classes** — `.o_data_row`, `.o_searchview_facet`, `.o_notebook`, `.o_statusbar_status`.
4. **Scoped visible text**; then **structural relative XPath** as a last resort.

Target	Resilient locator
Login fields	`input[name='login']`, `input[name='password']`, `button[type='submit']`
Customer Invoices	deep-link `action-account.action_move_out_invoice_type`
Clear facet	`.o_searchview_facet .o_facet_remove`
Open search menu	`.o_searchview_dropdown_toggler` → `.o-dropdown--menu`
Apply Posted	`get_by_role('menuitemcheckbox', name='Posted')`
Customer row	`tr.o_data_row` filtered by `td[name='invoice_partner_display_name']`
Posted state (untranslated)	`.o_statusbar_status .o_arrow_button_current` → `data-value='posted'`
Invoice Lines tab / table	`.o_notebook` → `.tab-pane.active .o_list_renderer`
Line fields	`[name='product_id'|'quantity'|'price_unit'|'tax_ids'|'price_subtotal'|'price_total']`

> **I18N ROBUSTNESS**
>
> Internal values such as `data-value='posted'` and field `name` attributes are not translated, whereas visible labels are. Where correctness must survive a non-English instance, the tool prefers the untranslated attribute over the label.

7. Network Interception & No-Sleep Synchronization

7.1 Authoritative data over the wire

Monetary and tax fields are computed server-side and rendered asynchronously (and rounded for display). Rather than scrape rendered text, the tool reads Odoo's own JSON-RPC response. All ORM reads ride the single endpoint `/web/dataset/call_kw` — there is no bare `/web/dataset/web_read` route, as `web_read` is a model method dispatched through `call_kw`. The predicate therefore matches that path and inspects the request body's `model` / `method`:

```
RPC_PATH = "/web/dataset/call_kw"   # web_read is dispatched through call_kw

def is_move_read(response) -> bool:
    if RPC_PATH not in response.url:
        return False
    params = (response.request.post_data_json or {}).get("params", {})
    return (params.get("model") == "account.move"
            and params.get("method") in ("web_read", "read"))

# Opening the invoice IS the trigger; we await its response (no sleep) and read
# the server's authoritative values straight from the JSON-RPC payload:
with page.expect_response(is_move_read) as info:
    row.locator("td.o_data_cell").first.click()
move = info.value.json()["result"][0]
# Per line:  tax_amount = price_total - price_subtotal   (both server-computed)
```

The opened invoice's payload nests the full line records under `invoice_line_ids` (`product_id`, `quantity`, `price_unit`, `tax_ids`, `price_subtotal`, `price_total`) and the header totals under `tax_totals`. Per-line tax is derived from server-computed values:

> **TAX DERIVATION**
>
> `tax_amount = price_total − price_subtotal`, summed and cross-checked against the header `tax_totals`. The DOM does not even render `price_total` as a default column — a concrete reason the network payload, not the screen, is the source of truth.

7.2 Eliminating sleeps

The automation contains **zero** `time.sleep` / `wait_for_timeout` calls. Synchronization is entirely signal-based:

Situation	Replacement
Server round-trip (filter, open, recompute)	`with page.expect_response(predicate):` around the triggering click — the response is the readiness signal.
Re-rendered content / value	Web-first auto-retrying assertions: `expect(loc).to_be_visible()`, `to_have_count(n)`.

Situation	Replacement
Element interaction	Playwright's built-in actionability checks (visible, stable, enabled) before every click/fill.
Session persistence	`launch_persistent_context` retains the session cookie across runs.

Network-idle waiting is deliberately avoided: an RPC/long-polling SPA may never idle, or may idle before the meaningful response arrives. Awaiting the specific response is both correct and faster.

8. Privacy Engineering: In-Memory PHI Sanitization

Billing data is treated as PHI. The guarantee — no customer or product identifier ever reaches a log sink — is delivered by a defense-in-depth, deny-list-first design. The deterministic layers run on every record; the ML layer is an optional backstop.

Layer	Mechanism	Property
0 · Discipline	Call sites log only safe scalars (counts, statuses, opaque ids) — never raw objects or PHI-bearing strings.	Prevention
1 · Deny-list	Exact PHI tokens captured at runtime (customer, products, invoice #) are removed by a compiled, case-insensitive matcher.	**Hard guarantee** · O(1)/ token
2 · Regex	Fast patterns for email, SSN, card, IP, URL, and Odoo invoice numbers.	Deterministic
3 · Presidio NER	Optional, lazy `en_core_web_sm` sweep (PERSON/ LOCATION) over residual free text; degrades open on failure.	Context-aware

8.1 The load-bearing detail

Python's logging renders the final string lazily as `record.msg % record.args`. Scrubbing only `record.msg` would leave PHI in `record.args`. The filter therefore renders once, scrubs, and clears `args` — and is attached to **handlers** so every sink is guarded, including records from third-party libraries:

```
class PhiRedactionFilter(logging.Filter):
    """Scrub every record before it reaches a sink; attach to HANDLERS."""

    def filter(self, record: logging.LogRecord) -> bool:
        # Render msg % args ONCE, scrub, then DROP args so the handler's later
        # getMessage() cannot re-introduce the original PHI values.
        record.msg = self._redactor.scrub(record.getMessage())
        record.args = ()
        if record.exc_info:                          # tracebacks bypass msg % args
            text = "".join(traceback.format_exception(*record.exc_info))
            record.exc_text = self._redactor.scrub(text)
            record.exc_info = None
        return True                                  # keep the now-clean record
```

> **WHY DENY-LIST, NOT ML, IS THE GUARANTEE**
>
> An exact-token deny-list is the only layer that can promise a specific name never appears; NER models are probabilistic. Cloud PII APIs are rejected outright — they would transmit log content off-host, violating the in-memory requirement. The tool also allow-lists operational, non-PHI values (e.g. the instance endpoint) so logs stay debuggable.

Exception and stack text bypass the `msg % args` path and are scrubbed explicitly; if sanitization itself ever failed, the filter fails closed (suppresses content) rather than risk a leak. The only file artifact the program writes is the final JSON extract.

9. HIPAA Compliance Mapping

SCOPE

HIPAA binds covered entities and business associates, not standalone software (45 CFR 160.103). This tool is a component; it becomes regulated when operated by/for such an entity handling PHI, which typically requires a Business Associate Agreement (164.502(e), 164.504(e)). Billing data is PHI because "payment for the provision of health care" is enumerated individually identifiable health information. Administrative (164.308) and Physical (164.310) safeguards are the operating organization's responsibility.

9.1 Technical safeguards realized in this tool

Control implemented here	CFR citation	Class
PHI handled only as transient in-memory objects; no raw PHI persisted	164.502(b); 164.312(a)(1)	Min. necessary / Access
PHI never logged (deny-list + regex scrub at the handler boundary)	164.502(b)	Min. necessary
Only the required billing fields are extracted (data minimization)	164.502(b); 164.514(d)	Min. necessary
Audit-friendly logging without PHI (counts, status, correlation)	164.312(b)	Audit controls
Unique service identity; least-privilege credentials from env/secret store	164.312(a)(2)(i)	**Required**
Authenticated session against the ERP	164.312(d)	Authentication
TLS to the instance (HTTPS) for data in transit	164.312(e)(1),(e)(2)(ii)	Transmission / Addr.
SHA-256 line fingerprint + header/line reconciliation	164.312(c)(1),(c)(2)	Integrity / Addr.
Optional Safe Harbor de-identification of the output	164.514(b)(2)	De-identification

"Addressable" does not mean optional (164.306(d)(3)): each must be implemented, or a documented, risk-based equivalent adopted. NIST/FIPS-validated encryption additionally triggers the HITECH breach safe harbor.

9.2 Safe Harbor de-identification (45 CFR 164.514(b)(2))

With `--deidentify`, the extract is transformed so it no longer contains the 18 Safe Harbor identifiers — after which it is, by regulation, no longer PHI. For billing data the relevant transforms are: names → salted-HMAC surrogate (A); dates → year only (C); account/invoice numbers → surrogate (J/R); and free-text descriptions are swept for residual identifiers.

The 18 identifiers (164.514(b)(2)(i))	
(A) Names	(J) Account numbers
(B) Geographic subdivisions < state	(K) Certificate / license numbers

The 18 identifiers (164.514(b)(2)(i))	
(C) Dates (except year); ages > 89	(L) Vehicle identifiers / plates
(D) Telephone numbers	(M) Device identifiers / serials
(E) Fax numbers	(N) Web URLs
(F) Email addresses	(O) IP addresses
(G) Social Security numbers	(P) Biometric identifiers
(H) Medical record numbers	(Q) Full-face photographs
(I) Health-plan beneficiary numbers	(R) Any other unique identifying number/characteristic/code

10. Data Model & Output Schema

The output is a typed, validated document (Pydantic). Per-line `tax_amount` is a computed field — always `total − subtotal` — so it cannot drift from its definition.

Field	Meaning
`metadata.*`	Tool version, UTC timestamp, source, instance URL, URL scheme, de-identification flag, integrity SHA-256.
`invoice.invoice_number`	e.g. `INV/2026/00001` (null for drafts).
`invoice.customer / state / currency`	Partner display name; status; ISO currency.
`invoice.amount_untaxed / amount_tax / amount_total`	Authoritative header totals from `tax_totals`.
`lines[].product / description`	Product display name; line label.
`lines[].quantity / unit_price`	Billed quantity; pre-tax unit price.
`lines[].taxes / subtotal / total / tax_amount`	Tax tag(s); net; tax-inclusive; derived per-line tax.
`checks.*`	Line count, line sums, and booleans reconciling line sums to the header.

10.1 Representative output


```
{
  "metadata": {
    "tool": "odoo-erp-extraction",
    "tool_version": "1.0.0",
    "extracted_at": "2026-05-29T02:14:46Z",
    "source": "json-rpc:account.move/web_read",
    "instance_url": "https://yr-solutions-srl.odoo.com",
    "url_scheme": "modern",
    "deidentified": false,
    "integrity_sha256": "dc149831b8a07a77b400fc28e7f19c7e583759cc04214c5df299a863b8c0c886"
  },
  "invoice": {
    "invoice_number": "INV/2026/00001",
    "customer": "Deco Addict",
    "invoice_date": "2026-05-20",
    "state": "posted",
    "currency": "EUR",
    "amount_untaxed": 690.0,
    "amount_tax": 122.82,
    "amount_total": 812.82,
    "source": "json-rpc:account.move/web_read"
  },
  "lines": [
    {
      "line_index": 0,
```

```
      "product": "Office Chair",
      "description": "Office Chair, ergonomic mesh",
      "quantity": 2.0,
      "unit_price": 120.5,
      "taxes": [
        "19%"
      ],
      "subtotal": 241.0,
      "total": 286.79,
      "currency": "EUR",
      "source": "json-rpc",
      "tax_amount": 45.79
    },
    {
      "line_index": 1,
      "product": "Standing Desk",
      "description": "Standing Desk, electric height-adjustable",
      "quantity": 1.0,
      "unit_price": 380.0,
      "taxes": [
        "19%"
      ],
      "subtotal": 380.0,
      "total": 452.2,
      "currency": "EUR",
      "source": "json-rpc",
      "tax_amount": 72.2
    },
    {
      "line_index": 2,
      "product": "A5 Notebook",
      "description": "A5 Notebook, dotted (pack)",
      "quantity": 10.0,
      "unit_price": 6.9,
      "taxes": [
        "7%"
      ],
      "subtotal": 69.0,
      "total": 73.83,
      "currency": "EUR",
      "source": "json-rpc",
      "tax_amount": 4.83
    }
  ],
  "checks": {
    "line_count": 3,
    "line_tax_sum": 122.82,
    "line_total_sum": 812.82,
    "line_tax_sum_equals_header": true,
    "line_total_sum_equals_amount_total": true
  }
}
```

11. Integrity & Verification

Each extract carries integrity evidence mapping to HIPAA 164.312(c):

- **SHA-256 fingerprint** over the canonical line set — any later alteration is detectable.
- **Header/line reconciliation** — the sum of per-line tax (and totals) is compared to the server's header `tax_totals`. These are two independent server computations; their agreement is strong evidence of a faithful extraction.

> **SELF-VERIFYING AFTER DE-IDENTIFICATION**
>
> The fingerprint always reflects the lines in the document at hand. When the Safe Harbor pass runs and redacts free-text, the fingerprint is **recomputed over the de-identified line set** — so both the original and the de-identified extract verify against their own contents, never a stale value.

On the verified run, both checks returned `TRUE` : `Σ line tax = 122.82 = amount_tax` and `Σ line total = 812.82 = amount_total`.

11.1 Automated tests

A 24-case offline suite covers the high-risk logic without a live instance:

Area	What is asserted
Sanitization	Deny-list removal, case-insensitivity, `args` clearing, exception scrubbing, conservative phone matching (dates/hashes spared), URL allow-listing, fail-closed behavior.
Interception	Relational coercion (dict/list encodings), section-row skipping, tax derivation, header totals from `tax_totals` , currency harvesting.
Models / integrity	Computed `tax_amount` , monetary rounding, reconciliation truth/false/none, deterministic fingerprint sensitivity.
De-identification	Identifier stripping, year-only dates, deterministic and non-destructive surrogates, salt sensitivity.

12. Operations & Runbook

The tool is a standard console script. Configuration resolves CLI flags → environment (`ODOO_*`) / `.env` → defaults; credentials are wrapped in `SecretStr` and never logged.

```
# Headless extraction (configuration via ODOO_* env vars / .env)
odoo-extract --customer "Deco Addict"

# Watch it run live during the demo
odoo-extract --headed --slow-mo 600

# Emit a HIPAA Safe Harbor de-identified extract
odoo-extract --deidentify --output output/invoice_extract.deidentified.json
```

Setting	Purpose
`ODOO_BASE_URL / --base-url`	Instance root URL.
`ODOO_USERNAME / --username`, `ODOO_PASSWORD`	Service-account login (password via env preferred).
`ODOO_TARGET_CUSTOMER / --customer`	Primary customer; aliases tried on miss.
`--headed / --slow-mo`	Show the browser; cosmetic per-action delay for demos.
`--deidentify`	Emit a Safe Harbor de-identified extract.
`--enable-presidio`	Enable the NER backstop (deny-list + regex always run).

Exit codes: `0` success · `1` unexpected · `2` extraction error · `3` customer not found — a stable contract for CI/orchestration.

13. Limitations, Assumptions & Future Work

- **Regulatory scope.** Software alone is not HIPAA-compliant; compliance is an organizational state requiring administrative and physical safeguards and, where applicable, a Business Associate Agreement.
- **Trial volatility.** odoo.com trials expire and may ship differing demo data; the seeding helper makes the demo deterministic and the locators are version-tolerant.
- **Conservative phone redaction.** The phone pattern intentionally favors precision (requires a `+` prefix or parenthesized area code) so it never clobbers dates, hashes, or timestamps; the exact deny-list remains the guarantee for names.
- **Presidio is optional.** The deterministic layers are always on; the NER backstop is opt-in and degrades open if the model is unavailable.
- **Future work.** Encryption-at-rest for any cached extract (FIPS-validated) to claim the HITECH breach safe harbor; pagination/batch extraction across many invoices; a structured audit-log sink; and an Expert Determination de-identification mode for richer datasets.

Appendix A — Verified Run Evidence

Console output of a live run against the trial. Note that only step messages, counts, and the integrity result appear — **no customer or product names** — while the instance URL and SHA-256 are preserved for operability:

```
2026-05-29 04:14:41  INFO  odoo_extractor.cli       odoo-extract v1.0.0 starting |
{"base_url": "https://yr-solutions-srl.odoo.com", "headless": true, "deidentify":
false, ...}
2026-05-29 04:14:42  INFO  odoo_extractor.client    Browser session started (headless=True).
2026-05-29 04:14:44  INFO  odoo_extractor.client    Submitted credentials for the configured
service account.
2026-05-29 04:14:45  INFO  odoo_extractor.client    Customer Invoices list loaded.
2026-05-29 04:14:45  INFO  odoo_extractor.client    Cleared 0 pre-existing search facet(s).
2026-05-29 04:14:45  INFO  odoo_extractor.client    Applied the 'Posted' status filter.
2026-05-29 04:14:45  INFO  odoo_extractor.client    Located a matching invoice row for the
target customer.
2026-05-29 04:14:46  INFO  odoo_extractor.client    Opened the invoice form (authoritative
record captured via JSON-RPC).
2026-05-29 04:14:46  INFO  odoo_extractor.extractor Extraction complete: 3 line(s) |
integrity tax_match=True total_match=True
2026-05-29 04:14:46  INFO  odoo_extractor.cli       Wrote 3 line(s) -> output/
invoice_extract.json | sha256=dc149831b8a0
```

The companion de-identified extract (`--deidentify`) renders the same invoice as `customer: "CUST-…"`, `invoice_number: "INV-…"`, `invoice_date: "2026"`, with free-text descriptions redacted — illustrating the Safe Harbor transform end-to-end.

Appendix B — References

Regulatory

- 45 CFR 160.103 — Definitions (PHI, covered entity, business associate).
- 45 CFR 164.312 — Technical safeguards.
- 45 CFR 164.502(b), 164.514(d) — Minimum necessary.
- 45 CFR 164.514(a)–(c) — De-identification (Safe Harbor & Expert Determination).
- 45 CFR 164.306(d) — Required vs Addressable specifications.
- HHS — Guidance to Render Unsecured PHI Unusable; NIST SP 800-52 / 800-111.

Tooling

- Playwright for Python — auto-waiting, web-first assertions, `expect_response`.
- Microsoft Presidio — Analyzer/Anonymizer; spaCy `en_core_web_sm`.
- Odoo web client (OWL) — `account.move` views; `/web/dataset/call_kw` JSON-RPC.
- WeasyPrint — CSS Paged Media rendering of this document; Pygments — code highlighting.

Prepared by Yashas Rattehalli · Senior AI Automation Engineer · Version 1.0.0 · May 29, 2026. This document and the accompanying software are confidential and provided for evaluation.